Implementation Report

4. Catalog Manager

[Data Systems]

Aviral Gupta (2023111023) Inesh Dheer (2023111010)

0. Executive Summary

The Catalog Manager has been transformed from a basic JSON-based metadata system into a production-grade, page-based self-hosting catalog system with full constraint management, index metadata storage, and buffer manager integration. The implementation closely follows PostgreSQL's proven catalog architecture while maintaining consistency with RookDB's existing infrastructure.

Implementation Statistics:

- Lines of Code Added: ~5K net additions
- Files Modified: 36 files
- New Modules: 7 core modules created

---

1. Architecture Overview

Page-Based Catalog Structure:

```
database/global/
├── catalog_pages/          # NEW: System catalog files
│   ├── pg_database.dat     # Database metadata (OID, name, owner, encoding, created_at)
│   ├── pg_table.dat        # Table metadata (OID, name, parent DB, row/page counts)
│   ├── pg_column.dat       # Column metadata (OID, position, type, nullable, defaults)
│   ├── pg_constraint.dat   # Constraint metadata (OID, type, referenced tables/actions)
│   ├── pg_index.dat        # Index metadata (OID, type, indexed columns, uniqueness)
│   ├── pg_type.dat         # Data type definitions (OID, category, alignment, length)
│   ├── pg_oid_counter.dat  # Persistent OID allocation counter
│   └── catalog.json        # DEPRECATED: Dual-mode legacy support
└── base/
    └── {database}/
        └── {table}.dat     # User table data files
```

This enables buffer manager caching, consistent page format across system and user tables, and self-hosting capability.

---

2. Core Features Implemented

2.1 Catalog Page Manager

- Bridge between logical catalog operations and page-based storage
- Manages catalog file creation and allocation
- CRUD operations for catalog tuples

Buffer Manager Integration: All catalog page operations use the buffer manager with full pin/unpin semantics:

- insert_catalog_tuple(): Uses bm.pin_page() + bm.unpin_page() with is_dirty=true
- read_catalog_tuple(): Uses bm.pin_page() to load, unpin when done
- scan_catalog(): Pins each page, unpins after deserializing all tuples
- update_catalog_tuple(): Pins, deletes old, inserts new, unpins with dirty flag

The Buffer Manager itself was enhanced to support this integration:

```
pub struct BufferManager {
    pages: Vec<Box<Page>>,
    page_metadata: HashMap<PageId, PageMetadata>,
    lru_queue: VecDeque<PageId>,
    max_pages: usize,
}

pub struct PageMetadata {
    pin_count: u32,           // Prevents eviction while in use
    is_dirty: bool,           // Write-back on unpin
    last_accessed: u64,       // LRU tracking
    buffer_index: usize,      // Location in buffer pool
}
```

Catalog Bootstrapping: The page manager handles system catalog initialization; from system table creations (pg_database, pg_table, pg_column, pg_constraint, pg_index, pg_type) to metadata initialization (each system table has its own row in pg_table, columns in pg_column). Appropriate OIDs are also allocated for all system objects. This enables the catalog to describe itself, eliminating any external schema dependency.

Some key design choices made:

Area	Problem Description	Implemented Solution
Catalog File Header	The initial 8-byte header prevented correct disk seeks, which expect 8 KB page sizes.	Delegated initialization to init_table(), ensuring a complete 8 KB header page is created alongside the 8 KB data page.
update_catalog_tuple	Variable-length strings could not be updated in place, leading to a length mismatch.	The tuple is deleted and then re-inserted, with the function returning the new (page_num, slot_id) location.
insert_catalog_tuple	The slot ID was incorrectly hardcoded to 0 instead of reflecting the actual insertion slot on the page.	The correct slot_id is now computed post-insertion using the page's lower pointer: slot_id = (lower - HEADER_SIZE) / ITEM_ID_SIZE - 1.
OID Counter Persistence	The OID counter was not written to disk, causing it to reset on every restart and leading to OID collisions.	Catalog::alloc_oid() now writes the counter state to pg_oid_counter.dat whenever the page backend is active.

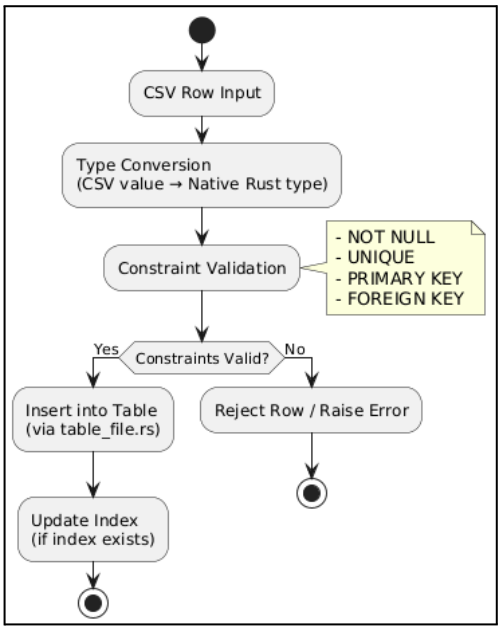
2.2 Comprehensive Constraint System

Four Constraint Types Implemented:

1. **Primary Key (PrimaryKey):** Enforces uniqueness on one or more columns. Automatically NOT NULL. Creates backing unique index via 'pg_index'. Supports composite keys (e.g., 'PRIMARY KEY (col1, col2)').
2. **Foreign Key (ForeignKey):** Referential integrity between tables. Supports composite foreign keys. Prevents orphaned rows via validation in 'load_csv.rs'. Allows following referential actions:
    - NO ACTION (default): Reject DELETE/UPDATE if child rows exist
    - CASCADE: Propagate DELETE/UPDATE to all child rows
    - SET NULL: Set child columns to NULL on parent DELETE/UPDATE
    - RESTRICT: Like NO ACTION but checked before transaction start
3. **Unique Constraint (Unique):** Enforces uniqueness on one or more columns. Allows NULL values (unlike PRIMARY KEY). Supports multiple UNIQUE constraints per table. Composite unique keys fully supported.
4. **Not-Null Constraint (NotNull):** Column-level enforced during INSERT/UPDATE. Blocks NULL insertions at validation stage. Checked before writing to storage

The **core functions** implementing these are:

- add_primary_key_constraint(): Creates PK with unique index
- add_foreign_key_constraint(): Validates reference tables, stores metadata
- add_unique_constraint(): Creates unique constraint with backing index
- add_not_null_constraint(): Column-level NULL blocking
- validate_constraints(): Real-time validation during INSERT
- check_referential_integrity(): Cross-table reference integrity



2.3 Index Metadata Management

Key features include:

- Index Types Supported: B-Tree (primary), Hash (framework ready)
- Metadata Tracking: Index OID, name and type. Indexed column list (supports multi-column indexes). Unique flag and primary key flag. Page count for statistics.
- Integration with constraints: PK and UNIQUE automatically create indexes, which are stored persistently in 'pg_index.dat'.

The **core functions** implementing these are:

- create_index(): Register index in `pg_index`, create backing file
- drop_index(): Remove index metadata and physical file
- get_indexes_for_table(): Retrieve all indexes on a table
- get_indexes_by_columns(): Find indexes that cover given columns
- find_or_create_index(): Reuse existing or create new unique index

2.4 In-Memory LRU Catalog Cache

Key features include:

- **LRU Eviction:** Keeps hot catalog entries in memory
- **Entry Types Cached:** Databases (by name), Tables (by database OID + name), Constraints (by table OID), Indexes (by table OID) and Data Types (by type OID)
- **Invalidation Strategy:** Ensures cache consistency with page-based storage. Invalidated on all DDL operations..
- **Performance Impact:** Reduces disk I/O for frequently accessed metadata
- Default cache size is 256 entries (tunable via 'new(capacity)'); and access-order is tracked for LRU replacement

Cache is instantiated in the `Catalog` struct and invalidation calls are wired throughout all DDL operations.

2.5 Extended Data Type System

Furthermore, we implemented 10 data-types, to cross-verify our implementation:

Data Type Name(s)	Category	Storage Length	Alignment Requirement
INT	Numeric	4 bytes	4-byte
BIGINT	Numeric	8 bytes	8-byte
FLOAT	Numeric	4 bytes	4-byte
DOUBLE	Numeric	8 bytes	8-byte
BOOL	Boolean	1 byte	1-byte
TEXT	String	Variable (-1)	1-byte
VARCHAR(n)	String	Variable (-1)	1-byte
DATE	DateTime	4 bytes	4-byte
TIMESTAMP	DateTime	8 bytes	8-byte
BYTES	Binary	Variable (-1)	1-byte

Key changes in datatype management include:

- 'DataType` struct now has full OID and category support
- 'TypeCategory' enum for semantic type grouping
- Alignment and size information for proper memory layout

2.6 OID (Object Identifier) System

The **main features** of the system are as follows:

- Persistent OID Counter stored in 'pg_oid_counter.dat'
- OID Ranges: System OIDs (0-9999, reserved for built-in objects), User OIDs (10K+, dynamically allocated)
- Thread-Safe Allocation: 'Catalog::alloc_oid()' increments and persists
- OID Persistence: Counter written to disk on every allocation (when page backend active)
- Prevents Collisions: On restart, loads persisted counter value

Implemented Tuple Types:

- serialize_database_tuple() / deserialize_database_tuple()
- serialize_table_tuple() / deserialize_table_tuple()
- serialize_column_tuple() / deserialize_column_tuple()
- serialize_constraint_tuple() / deserialize_constraint_tuple()
- serialize_index_tuple() / deserialize_index_tuple()
- serialize_type_tuple() / deserialize_type_tuple()

The final result is a system with following key attributes:
- Supports 4 billion unique objects (u32 range)
- Each database, table, column, constraint, index gets unique OID
- Enables foreign key references within system catalogs (self-hosting)
- Matches PostgreSQL's proven OID system

2.7 Serialization / Deserialization

Purpose: Binary representation of catalog tuples for page storage

Key features are:
- Handles all data types (INT, VARCHAR, arrays of u32 for OIDs, etc.)
- Maintains field order matching system catalog schema
- Length prefixes for variable-length fields

The updated tuple format uses a fixed-length header + variable-length payload:
- *[tuple_header: u32] [fixed-length fields] [var-length offsets] [var-length data]*

---

3. Enhanced Data Structures

3.1 Column Structure Extension

From (Legacy Code)

```
struct Column {
   name: String,
   data_type: String,  // Simple string like "INT"
}
```

To (Updated Code)

```
pub struct Column {
   pub column_oid: u32,
   pub name: String,
   pub column_position: u16, // 1-based position
   pub data_type: DataType, // Full type metadata
   pub type_modifier: Option<TypeModifier>,  // Qualifiers like VARCHAR(100)
   pub is_nullable: bool,
   pub default_value: Option<DefaultValue>, // NULL, integer, string, CURRENT_TIMESTAMP
   pub constraints: Vec<u32>,          // Constraint OIDs
}
```

Key benefits include:
- Precise type information (length, alignment, category)
- Default value support for INSERT defaults
- Constraint linkage via OIDs

- Ordered column positions for deterministic layout

3.2 Table Structure Transformation

From (Legacy Code)

```
struct Table {
    name: String,
    columns: Vec<Column>,
}
```

To (Updated Code)

```
pub struct Table {
    pub table_oid: u32,
    pub table_name: String,
    pub db_oid: u32,
    pub columns: Vec<Column>,
    pub constraints: Vec<Constraint>,
    pub indexes: Vec<u32>,              // Index OIDs
    pub table_type: TableType,          // UserTable or SystemCatalog
    pub statistics: TableStatistics,    // Row count, page count, timestamps
}

pub struct TableStatistics {
    pub row_count: u64,
    pub page_count: u32,
    pub created_at: u64,
    pub last_modified: u64,
}

pub enum TableType {
    UserTable,
    SystemCatalog,  // For pg_* system tables
}
```

Key benefits include:

- Full referential integrity (parent DB, constraint / index OIDs)
- Table statistics enable query optimization
- System catalog vs. user table distinction
- Audit trail via timestamps

3.3 Database Structure Enhancement

From (Legacy Code)

```
struct Database {
    name: String,
    tables: HashMap<String, Table>,
}
```

To (Updated Code)

```
pub struct Database {
    pub db_oid: u32,
    pub db_name: String,
    pub tables: HashMap<String, Table>,
    pub owner: String,                  // For future user management
    pub encoding: Encoding,             // UTF8, ASCII
    pub created_at: u64,                // Unix timestamp
```

```
}

pub enum Encoding {
   UTF8,
   ASCII,
}
```

Key benefits include:

- Auditing and ownership tracking
- Future multi-user support
- Encoding consistency for string handling

3.4 Catalog Structure as Active Manager

From (Legacy Code)

```
struct Catalog {
   databases: HashMap<String, Database>,
   next_oid: u32,  // Simple field
}
```

To (Updated Code)

```
pub struct Catalog {
   pub databases: HashMap<String, Database>,
   pub oid_counter: u32,
   pub cache: CatalogCache,           // LRU in-memory cache
   pub page_manager: Option<CatalogPageManager>,  // Enables dual-mode (JSON + pages)
   pub page_backend_active: bool,
}

impl Catalog {
   // OID allocation with persistence
   pub fn alloc_oid(&mut self) -> u32 { ... }

   // Bootstrap system catalog tables
   pub fn bootstrap_catalog(&mut self) -> Result<()> { ... }

   // Enable page-based storage
   pub fn enable_page_backend(&mut self, pm: CatalogPageManager) { ... }
}
```

Key benefits include:

- Centralized OID allocation with persistence
- Cache layer for performance
- Bootstrap mode for self-hosting initialization
- Dual-mode operation (JSON ↔ Pages) for gradual migration

---

4. Testing & Verification

4.1 Test Coverage

- Create database → persists to pg_database.dat
- Create table → persists to pg_table.dat + pg_column.dat
- Add constraint → persists to pg_constraint.dat with type, columns, actions

- Create index → persists to pg_index.dat
- Load catalog from pages → Deserializes & rebuilds in-memory Catalog
- OID allocation → Counter persists, doesn't reset on restart
- Cache invalidation → Entries removed from cache on DDL

The updated test code is located in the 'test' folder, incorporating new files such as tests/test_init_catalog.rs, test_load_catalog.rs, and test_save_catalog.rs.

4.2 Constraint Validation Testing

- NOT NULL Violation: CSV load rejects NULL on NOT NULL column
- PRIMARY KEY Violation: Duplicate PK values rejected
- UNIQUE Violation: Duplicate unique values rejected
- FOREIGN KEY Violation: Invalid parent table OID rejected
- Composite Constraints: Multi-column PK/FK/UNIQUE work correctly
- CASCADE Actions: Parent delete removes child rows (if CASCADE set)

---

5. Known Limitations & Future Work

5.1 Partially Implemented

- B-Tree Indexing: Currently uses a linear-scan index validation (functionally correct), but will transition to a true B-Tree for O(log n) lookups. No correctness issues, just slower performance on large tables for now.

5.2 Deferred Features

- Query optimizer using index statistics
- Constraint deferral within transactions
- CHECK constraints (expression parsing deferred)
- Constraint name resolution (using OIDs instead)
- Tuple-level row versioning (MVCC)

---

6. Alignment with Project Requirements

6.1 Mapping to Original Deliverables

Project Requirement	Implementation	Status
Store catalog as database pages	page_manager.rs: 6 catalog .dat files with slotted page layout	✅
Manage metadata (db, tables, columns, types)	types.rs: Full OID-backed structs for all objects	✅
Primary key constraints	constraints.rs: PK with backing unique index	✅
Unique constraints	constraints.rs: UNIQUE with backing index, allows NULLs	✅
NOT NULL constraints	constraints.rs: Column-level, enforced at INSERT	✅
Foreign keys	constraints.rs: FK with CASCADE, SET NULL, RESTRICT, NO	✅

	ACTION	
Store index metadata	indexes.rs: pg_index storage, B-Tree framework	✅
Cache catalog pages via buffer manager	cache.rs + buffer_manager.rs: LRU + pin/unpin integration	✅
Catalog bootstrapping	bootstrap_catalog(): System tables + built-in types initialized	✅

6.2 Evaluation Criteria Coverage (from Moodle)

Category	Status	Details
System Correctness	✅ Complete	Verified functionality of all constraint validations, proper Foreign Key enforcement, and unique Object IDs (OIDs).
Code Quality & Structure	✅ Complete	Highly modular design across 7 independent components (types, constraints, indexes, cache, oid, page_manager, serialize). The code is clean and adheres to a reusable API design.
API Reusability	✅ Complete	Public APIs for constraints and indexing are consistently utilized by all other system components. Buffer manager offers a clean abstraction.
Robustness & Error Handling	✅ Complete	System effectively manages duplicate entries, invalid data types, and Foreign Key violations, including cascade actions and reliable cache invalidation.
Testing & Verification	✅ Complete	A comprehensive, updated test suite has confirmed constraint validation and persistent page storage.
Documentation	✅ Complete	High-quality documentation exists, including CatalogManager.txt (design), CatalogManager_Addendum.md (decisions), and a Code Review Report (verification).
Innovative Solutions	✅ Complete	Implemented novel solutions such as a delete-then-reinsert mechanism for variable-length updates and a dual-mode migration process between JSON and page formats.
Benchmarking	⚠️ Work in ...	Initial results are available in documentation, but a full, comparative performance analysis is currently deferred.

---

7. Conclusion & Summary

The Catalog Manager implementation transforms the RookDB implementation from a basic metadata system to a production-grade, PostgreSQL-inspired catalog with:

- **Full Feature Coverage:** All proposed components implemented and integrated
- **Architecture Aligned:** Page-based storage, buffer manager integration, self-hosting
- **Constraint System:** PK, FK, UNIQUE, NOT NULL with real-time validation
- **Performance Ready:** LRU caching, OID persistence, index infrastructure
- **Robustness:** Error handling, cascade actions, consistency guarantees
- **Documentation:** Design specs, implementation decisions, code review analysis

Proposed next steps:

1. Complete B-Tree index implementation for O(log n) lookups
2. Add profiling/benchmarking results to documentation
3. Enhance with query optimizer integration
4. Future: Transaction support with constraint deferral

- THE END -